

JobSheet - Week 7 – Multi-Thread

Nama : Raza Mulki Firdaus
NIM : 251524120
Kelas : 1D
Repo GitHub : <https://github.com/zaaraza/Teknik-Pemrograman->

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W7_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W7_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Transfer Fulus

Soal Praktikum
Instruksi Pengerjaan: a. Analisis Masalah: Perhatikan source code di bawah ini, masalah apa yang Anda temukan? Berikan penjelasannya pada setiap bagian source code dengan menyeluruh berdasarkan materi yang telah diperoleh pada kelas teori! b. Solusi Terhadap Masalah: Tuliskan solusi berupa source code dan penjelasan solusi terhadap masalah tersebut! Berikan penjelasannya pada setiap bagian source code! Source Code: <pre>class Account { int balance = 150; }</pre>

```

public class TransferFulus {
    public static void main(String[] args) throws InterruptedException {
        Account acc1 = new Account();
        Account acc2 = new Account();

        // Thread 1: Menjumlahkan/ transfer fulus dari acc1 ke acc2
        Thread t1 = new Thread(() -> {
            synchronized (acc1) { // Berikan komentar apa yang dilakukan pada blok ini?
                System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
                try { Thread.sleep(100); } catch (Exception e) {} // Simulasi dengan
memberikan jeda. Mengapa diperlukan Exception?

                synchronized (acc2) { // Berikan komentar apa yang dilakukan pada blok ini?
                    System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
                    acc2.balance += acc1.balance;
                }
            }
        });

        // Thread 2: Menjumlahkan/ transfer fulus dari acc2 ke acc1
        Thread t2 = new Thread(() -> {
            synchronized (acc2) { // Berikan komentar apa yang dilakukan pada blok ini?
                System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
                try { Thread.sleep(100); } catch (Exception e) {}

                synchronized (acc1) { // Berikan komentar apa yang dilakukan pada blok ini?
                    System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
                    acc1.balance += acc2.balance;
                }
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("--- HASIL AKHIR ---");
        System.out.println("Saldo Akhir acc1: " + acc1.balance);
        System.out.println("Saldo Akhir acc2: " + acc2.balance);
    }
}

```

Source Code

```
1 class Account {
2     int balance = 150;
3 }
4
5 public class Transferulus {
6     public static void main(String[] args) throws InterruptedException {
7         Account acc1 = new Account();
8         Account acc2 = new Account();
9
10        // Thread 1: Transfer dari acc1 ke acc2
11        Thread t1 = new Thread() -> {
12            // SOLUSI: selalu mengunci acc1 dulu, baru acc2
13            synchronized (acc1) {
14                System.out.println("Thread 1: Mengunci acc1...");
15                try { Thread.sleep(100); } catch (Exception e) { e.printStackTrace(); }
16
17                synchronized (acc2) {
18                    System.out.println("Thread 1: Berhasil mengunci acc2. Melakukan transfer...");
19                    acc2.balance += acc1.balance;
20                    acc1.balance = 0;
21                }
22            }
23        };
24
25        // Thread 2: Transfer dari acc2 ke acc1
26        Thread t2 = new Thread() -> {
27            // Urutan synchronized kunci disamakan dengan Thread 1 (acc1 dulu baru acc2)
28            synchronized (acc1) {
29                System.out.println("Thread 2: Mengunci acc1...");
30                try { Thread.sleep(100); } catch (Exception e) { e.printStackTrace(); }
31
32                synchronized (acc2) {
33                    System.out.println("Thread 2: Berhasil mengunci acc2. Melakukan transfer...");
34                    acc1.balance += acc2.balance;
35                    acc2.balance = 0;
36                }
37            }
38        };
39
40        t1.start();
41        t2.start();
42
43        t1.join();
44        t2.join();
45
46        System.out.println("--- HASIL AKHIR ---");
47        System.out.println("Saldo Akhir acc1: " + acc1.balance);
48        System.out.println("Saldo Akhir acc2: " + acc2.balance);
49    }
50 }
```

Screenshot Hasil

```
Thread 1: Mengunci acc1...
Thread 1: Berhasil mengunci acc2. Melakukan transfer...
Thread 2: Mengunci acc1...
Thread 2: Berhasil mengunci acc2. Melakukan transfer...
--- HASIL AKHIR ---
Saldo Akhir acc1: 300
Saldo Akhir acc2: 0
```

Penjelasan

Soal ini permasalahannya adalah berkaitan dengan deadlock, dimana ada dua thread saling menunggu resource yang dipegang thread lain, sehingga tidak ada satupun yang bisa melanjutkan eksekusi. Pada kasus ini resourcesnya adalah acc1 dan acc2 dimana thread 1 me-lock acc 1 dan ingin mengakses acc2, namun di thread 2 acc2 dilock dan ingin mengakses acc1. Karena tidak ada jalan keluar, keduanya saling menutup diri maka untuk mengatasinya adalah dengan menggunakan synchronized. Maka setelah digunakan synchronized akhirnya keduanya bisa saling mengakses.

Kedua thread harus dipaksa untuk meminta kunci dengan urutan yang sama (misalnya: selalu kunci acc1 dulu, baru kemudian acc2). Dengan urutan yang seragam, jika Thread 1 sudah memegang kunci acc1, maka Thread 2 akan tertahan di pintu pertama dan tidak akan sempat mengunci acc2. Ini mencegah terjadinya situasi saling sandera.

Nama Teman Hal yang Dibantu (Opsional)

No. 2 Resto Ayam

Soal Praktikum

Instruksi Pengerjaan:

- Analisis Masalah:** Perhatikan source code di bawah ini, masalah apa yang Anda temukan? Berikan penjelasannya pada setiap bagian source code dengan menyeluruh berdasarkan materi yang telah diperoleh pada kelas teori!
- Solusi Terhadap Masalah:** Tuliskan solusi berupa source code dan penjelasan solusi terhadap masalah tersebut! Berikan penjelasannya pada setiap bagian source code!

Source Code:

```
class Resto {
    private int chickenStock = 100;

    public void serveCustomer(String cashierName) {
        if (chickenStock > 0) {
            try { Thread.sleep(10); } catch (InterruptedException e) {}

            chickenStock--;
            System.out.println(cashierName + " berhasil menjual 1 ayam. Sisa stok: " +
chickenStock);
        } else {
            System.out.println(cashierName + " gagal: Stok Habis!");
        }
    }

    public int getRemainingStock() {
        return chickenStock;
    }
}

public class RestoSimulasi {
    public static void main(String[] args) throws InterruptedException {
        Resto ayamJuicyLuicyGallagher = new Resto();

        Runnable task = () -> {
            for (int i = 0; i < 40; i++) {

ayamJuicyLuicyGallagher.serveCustomer(Thread.currentThread().getName());
            }
        };

        Thread kasir1 = new Thread(task, "Kasir-A");
        Thread kasir2 = new Thread(task, "Kasir-B");
    }
}
```

```
Thread kasir3 = new Thread(task, "Kasir-C");
```

```
kasir1.start();
```

```
kasir2.start();
```

```
kasir3.start();
```

```
kasir1.join();
```

```
kasir2.join();
```

```
kasir3.join();
```

```
System.out.println("--- HASIL AKHIR STOK: " +  
    ayamJuicyLuicyGallagher.getRemainingStock() + " ---");  
}  
}
```

Source Code

```
1 class Resto {  
2     // Stok awal ayam  
3     private int chickenStock = 100;  
4  
5     // Menambahkan 'synchronized' agar hanya satu thread (kasir)  
6     // yang bisa menjalankan method ini dalam satu waktu.  
7     public synchronized void serveCustomer(String cashierName) {  
8         if (chickenStock > 0) {  
9             // Simulasi proses transaksi  
10            try {  
11                Thread.sleep(10);  
12            } catch (InterruptedException e) {  
13                Thread.currentThread().interrupt();  
14            }  
15  
16            chickenStock--;  
17            System.out.println(cashierName + " berhasil menjual 1 ayam. Sisa stok: " + chickenStock);  
18        } else {  
19            System.out.println(cashierName + " gagal: Stok Habis!");  
20        }  
21    }  
22  
23    public int getRemainingStock() {  
24        return chickenStock;  
25    }  
26 }  
27  
28 public class RestoSimulasi {  
29     public static void main(String[] args) throws InterruptedException {  
30         Resto ayamJuicyLuicyGallagher = new Resto();  
31  
32         // Setiap kasir (thread) akan mencoba melayani 40 pelanggan  
33         Runnable task = () -> {  
34             for (int i = 0; i < 40; i++) {  
35                 ayamJuicyLuicyGallagher.serveCustomer(Thread.currentThread().getName());  
36             }  
37         };  
38  
39         // Membuat 3 thread kasir  
40         Thread kasir1 = new Thread(task, "Kasir-A");  
41         Thread kasir2 = new Thread(task, "Kasir-B");  
42         Thread kasir3 = new Thread(task, "Kasir-C");  
43  
44         // Memulai simulasi  
45         kasir1.start();  
46         kasir2.start();  
47         kasir3.start();  
48  
49         // Menunggu semua kasir selesai bekerja sebelum mencetak hasil akhir  
50         kasir1.join();  
51         kasir2.join();  
52         kasir3.join();  
53  
54         System.out.println("\n=====");  
55         System.out.println("--- HASIL AKHIR STOK: " + ayamJuicyLuicyGallagher.getRemainingStock() + " ---");  
56         System.out.println("=====");  
57     }  
58 }
```

Screenshot Hasil

```

Kasir-A berhasil menjual 1 ayam. Sisa stok: 78
Kasir-A berhasil menjual 1 ayam. Sisa stok: 77
Kasir-A berhasil menjual 1 ayam. Sisa stok: 76
Kasir-A berhasil menjual 1 ayam. Sisa stok: 75
Kasir-B berhasil menjual 1 ayam. Sisa stok: 74
Kasir-B berhasil menjual 1 ayam. Sisa stok: 73
Kasir-B berhasil menjual 1 ayam. Sisa stok: 72
Kasir-B berhasil menjual 1 ayam. Sisa stok: 71
Kasir-B berhasil menjual 1 ayam. Sisa stok: 70
Kasir-C berhasil menjual 1 ayam. Sisa stok: 69
Kasir-C berhasil menjual 1 ayam. Sisa stok: 68

```

Saat stok habis

```

Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!

```

```

=====
--- HASIL AKHIR STOK: 0 ---
=====

```

Penjelasan

Masalah utama pada kode awal Resto Ayam adalah Race Condition. Ini adalah situasi di mana beberapa thread mencoba mengakses dan mengubah data yang sama secara bersamaan, sehingga hasil akhirnya bergantung pada urutan eksekusi thread yang tidak terduga.

Solusi pada kasus ini adalah dengan menggunakan kata kunci synchronized. Ini bertujuan untuk mengunci akses ke shared resource (stok ayam). Hal ini memaksa setiap thread kasir untuk mengantre sehingga proses pemeriksaan dan pengurangan stok terjadi secara berurutan (sequential). Dengan demikian, masalah Race Condition dapat dihindari karena tidak ada dua thread yang bisa memodifikasi variabel stok pada saat yang bersamaan.

Nama Teman Hal yang Dibantu (Opsional)

-

No. 3 Penjumlahan Paralel

Soal Praktikum

Instruksi Pengerjaan:

Buatlah suatu program yang bertujuan untuk melakukan penjumlahan secara paralel dengan ketentuan sebagai berikut:

- Menerima Input: (1) Jumlah Thread; (2) Angka Akhir
- Contoh: Menjumlahkan angka dari 1 sampai "Angka Akhir". Jika "Jumlah Thread" = 4, "Angka Akhir" = 1000, maka:
 - Thread 1: Menjumlahkan 1 - 250
 - Thread 2: Menjumlahkan 251 - 500
 - Thread 3: Menjumlahkan 501 - 750
 - Thread 4: Menjumlahkan 751 - 1000
- Hasil akhir diperoleh dari hasil parsial (dari masing-masing thread) yang dijumlahkan menjadi total akhir.
- Perhatikan Thread Safety, Mekanisme Pembagian Tugas (Divide and Conquer), dan Synchronization agar hasil akhir sesuai!
- Pada program, tampilkan output:
 - Tugas apa yang dikerjakan oleh masing-masing thread
 - Hasil parsial dari masing-masing thread
 - Hasil akhir yang merupakan penjumlahan dari semua hasil parsial
- Berikan penjelasan yang lengkap untuk setiap bagian pentingnya!

Source Code


```

1  import java.util.Scanner;
2
3  public class PenjumlahanParalel {
4
5      // Variabel untuk menyimpan total akhir, menggunakan volatile agar
6      // perubahan dari satu thread langsung terlihat oleh thread lain
7      private static long totalAkhir = 0;
8
9      // Objek kunci (lock) khusus untuk sinkronisasi saat menjumlahkan hasil parsial
10     private static final Object lock = new Object();
11
12     public static void main(String[] args) throws InterruptedException {
13         Scanner scanner = new Scanner(System.in);
14
15         // --- INPUT ---
16         System.out.print("Masukkan Jumlah Thread : ");
17         int jumlahThread = scanner.nextInt();
18
19         System.out.print("Masukkan Angka Akhir : ");
20         int angkaAkhir = scanner.nextInt();
21
22         scanner.close();
23
24         // --- PEMBAGIAN TUGAS (DIVIDE AND CONQUER) ---
25         // Hitung berapa angka yang ditangani setiap thread
26         int ukuranBagian = angkaAkhir / jumlahThread;
27
28         // Array untuk menyimpan semua objek Thread
29         Thread[] threads = new Thread[jumlahThread];
30
31         System.out.println("\n--- PROSES PENJUMLAHAN PARALEL ---");
32
33         // --- PEMBUATAN DAN PENJALANAN THREAD ---
34         for (int i = 0; i < jumlahThread; i++) {
35             // Tentukan batas bawah dan batas atas untuk setiap thread
36             int batasBawah = (i * ukuranBagian) + 1;
37             // Thread terakhir mengambil sisa angka jika angkaAkhir tidak habis dibagi
38             int batasAtas = (i == jumlahThread - 1) ? angkaAkhir : (i + 1) * ukuranBagian;
39
40             // Nama thread dimulai dari "Thread-1"
41             String namaThread = "Thread-" + (i + 1);
42
43             // Buat thread baru menggunakan lambda, dengan final variable agar bisa diakses di dalam lambda
44             final int finalBawah = batasBawah;
45             final int finalAtas = batasAtas;
46
47             threads[i] = new Thread(() -> {
48                 // Tampilkan tugas yang dikerjakan thread ini
49                 System.out.println(Thread.currentThread().getName() +
50                     " : Menjumlahkan " + finalBawah + " - " + finalAtas);
51
52                 // Hitung hasil parsial untuk rentang angka yang diberikan
53                 long hasilParsial = 0;
54                 for (int j = finalBawah; j <= finalAtas; j++) {
55                     hasilParsial += j;
56                 }
57
58                 // Tampilkan hasil parsial dari thread ini
59                 System.out.println(Thread.currentThread().getName() +
60                     " : Hasil Parsial = " + hasilParsial);
61
62                 // --- THREAD SAFETY / SINKRONISASI ---
63                 // Gunakan blok synchronized saat menambahkan hasil parsial ke totalAkhir
64                 // agar tidak terjadi Race Condition (hanya satu thread yang boleh
65                 // memodifikasi totalAkhir pada satu waktu)
66                 synchronized (lock) {
67                     totalAkhir += hasilParsial;
68                 }
69
70                 }, namaThread); // Nama thread diberikan langsung di konstruktor
71
72             // Jalankan thread
73             threads[i].start();
74         }
75
76         // --- SINKRONISASI THREAD UTAMA ---
77         // join() memastikan thread utama menunggu semua thread selesai
78         // sebelum mencetak hasil akhir
79         for (int i = 0; i < jumlahThread; i++) {
80             threads[i].join();
81         }
82
83         // --- OUTPUT HASIL AKHIR ---
84         System.out.println("\n--- HASIL AKHIR ---");
85         System.out.println("Total Penjumlahan 1 sampai " + angkaAkhir +
86             " menggunakan " + jumlahThread + " thread = " + totalAkhir);
87     }
88 }

```

Screenshot Hasil	
	<pre> Masukkan Jumlah Thread : 4 Masukkan Angka Akhir : 1000 --- PROSES PENJUMLAHAN PARALEL --- Thread-3 : Menjumlahkan 501 - 750 Thread-1 : Menjumlahkan 1 - 250 Thread-4 : Menjumlahkan 751 - 1000 Thread-2 : Menjumlahkan 251 - 500 Thread-3 : Hasil Parsial = 156375 Thread-4 : Hasil Parsial = 218875 Thread-2 : Hasil Parsial = 93875 Thread-1 : Hasil Parsial = 31375 --- HASIL AKHIR --- Total Penjumlahan 1 sampai 1000 menggunakan 4 thread = 500500 </pre>
Penjelasan	
Program ini menerapkan konsep penjumlahan paralel menggunakan mekanisme Divide and Conquer, di mana rentang angka dari 1 hingga Angka Akhir dibagi secara merata ke sejumlah thread sesuai input pengguna. Setiap thread mendapat bagian rentang angkanya sendiri, menghitung hasil parsial secara mandiri dan bersamaan, lalu menyumbangkan hasilnya ke total akhir. Untuk menjaga Thread Safety, digunakan blok synchronized (lock) saat setiap thread menambahkan hasil parsialnya ke variabel totalAkhir. Blok ini memastikan hanya satu thread yang boleh mengakses dan memodifikasi totalAkhir pada satu waktu, sehingga tidak terjadi Race Condition yang bisa menyebabkan nilai total salah atau tidak konsisten. Sinkronisasi antar thread dengan thread utama (main) dilakukan menggunakan join(). Tanpa join(), thread utama bisa langsung mencetak hasil akhir sebelum semua thread selesai menghitung, sehingga nilai yang ditampilkan bisa salah atau nol. Urutan output dari masing-masing thread terlihat tidak berurutan (acak) karena thread-thread tersebut memang berjalan secara arallel dan bersaing mendapatkan akses ke System.out.println(). Ini adalah perilaku normal dari multithreading dan justru membuktikan bahwa program benar-benar berjalan secara konkuren. Meskipun demikian, hasil akhir selalu konsisten dan benar karena mekanisme sinkronisasi telah diterapkan dengan tepat.	
Nama Teman Hal yang Dibantu (Opsional)	
-	